The Full Transformer (Encoder-Decoder)

Document 3 of 6 — Building a GPT from Scratch

By now you understand the building blocks: scaled dot-product attention, multi-head attention, feed-forward networks, residual connections, and layer normalization. Documents 01 and 02 showed how these components form a transformer block. This document steps back to the original architecture from "Attention Is All You Need" (Vaswani et al., 2017) and explains how those blocks are arranged into a full encoder-decoder model for machine translation.

We are not building this architecture ourselves — our BabyGPT will use only the decoder half. But understanding the full picture is necessary to appreciate what GPT keeps, what it drops, and why.

1 The Sequence-to-Sequence Problem

Machine translation takes a sentence in one language and produces the equivalent sentence in another. More broadly, many problems have this shape: an input sequence goes in, and a different-length output sequence comes out. Summarization (long document in, short summary out), speech-to-text (audio frames in, words out), and even question answering (context + question in, answer out) are all sequence-to-sequence tasks.

The **encoder-decoder** paradigm for handling this predates transformers. Sutskever et al. (2014) used two RNNs: an encoder RNN reads the input sequence token by token and produces a final hidden state (a single vector), then a decoder RNN generates the output sequence from that vector one token at a time.

The problem is immediate: compressing an arbitrarily long input sentence into a single fixed-size vector is a severe bottleneck. Information is inevitably lost, and the effect gets worse as the input sequence grows. A 50-word sentence and a 5-word sentence both get squeezed into the same-size vector.

Bahdanau et al. (2015) solved this by introducing **attention** into the encoder-decoder RNN. Instead of using only the encoder's final hidden state, the decoder at each time step computes a weighted sum over *all* of the encoder's hidden states. The decoder learns to focus on the relevant parts of the input as it generates each output token. This was the original attention mechanism.

The transformer takes this idea and makes it the entire architecture. There are no recurrent connections at all — just attention, feed-forward networks, and residual connections. The encoder uses self-attention to build rich representations of the input, the decoder uses self-attention to build representations of the output so far, and cross-attention lets the decoder read from the encoder. The result is a model that is more parallelizable and more powerful than RNN-based approaches.

2 The Transformer Encoder

The encoder is a stack of $N = 6$ identical layers. Each encoder layer has two sublayers:

1. **Multi-head self-attention** (bidirectional — no causal mask)
2. **Position-wise feed-forward network**

Both sublayers are wrapped with a residual connection and followed by layer normalization (the original paper uses post-norm: `LayerNorm(x + Sublayer(x))`).

The structure of the full encoder:

```
Input Tokens -> Embedding + Positional Encoding
    -> [EncoderLayer x N]
    -> Encoder Output (B, T_src, d_model)
```

B is the batch size (number of sentences processed in parallel), T_{src} is the length of the source sequence (number of input tokens after padding), and d_{model} is the embedding dimension — the width of every hidden representation in the model (512 in the base transformer).

The critical difference from the decoder blocks in Document 02: **there is no causal mask**. Every position can attend to every other position. This is **bidirectional** attention — position 5 can look at position 10, and position 10 can look at position 5.

Why bidirectional? The encoder processes the entire input at once. When translating "The cat sat on the mat" from English to German, the encoder already has the complete input sentence. There is no reason to hide any positions from each other. The word "cat" should be able to use context from "mat" at the end of the sentence, and vice versa. Restricting the encoder to only look leftward (as a causal mask does) would throw away useful information for no benefit.

The only masking in the encoder is a **source mask** (also called a padding mask). When training on batches of sentences, shorter sentences are padded to the length of the longest sentence in the batch. We mask out these padding positions so they do not contribute to attention — they are not real tokens and should not influence the representation of actual tokens.

The output of the encoder is a tensor of shape `(B, T_src, d_model)`: a sequence of contextualized representations, one per input position. After passing through 6 layers of bidirectional self-attention, each position's representation encodes information from the entire input sequence. Position 3's output vector is not just about the token at position 3 — it is a representation of that token in the context of all other tokens.

3 The Transformer Decoder

The decoder is also a stack of $N = 6$ identical layers, but each decoder layer has **three** sublayers (not two):

1. **Masked multi-head self-attention** (with causal mask — same as our GPT blocks from Document 02)
2. **Multi-head cross-attention** (queries from the decoder, keys and values from the encoder output)
3. **Position-wise feed-forward network**

All three sublayers have residual connections and layer normalization.

The decoder generates output tokens one at a time during inference. When translating to German, it generates "Die", then "Katze", then "saß", and so on. The causal mask in the first sublayer ensures that when generating token at position t, the decoder can only attend to positions 0 through t (no peeking at future tokens it has not generated yet). During training, **teacher forcing** feeds the correct target sequence shifted by one position, but the causal mask still prevents the model from cheating by looking ahead.

The structure of the full decoder:

```
Target Tokens -> Embedding + Positional Encoding
    -> [DecoderLayer x N]
```

```
-> Linear -> Softmax -> Output Probabilities
```

4 Cross-Attention: How the Decoder Reads the Encoder

Cross-attention is the mechanism that connects encoder and decoder. It is the reason the decoder can generate output that is conditioned on the input — without it, the decoder would just be a language model with no knowledge of the source sentence.

Cross-attention uses the exact same multi-head attention mechanism from Document 01, but the queries, keys, and values come from different places:

- **Queries (Q):** from the decoder's current state (the output of the masked self-attention sublayer)
- **Keys (K) and Values (V):** from the encoder's final output

Concretely, for decoder layer l:

$$\text{CrossAttn}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where $Q = H_{\text{dec}}^{(l)} W^Q$, $K = H_{\text{enc}} W^K$, $V = H_{\text{enc}} W^V$.

$H_{\text{dec}}^{(l)}$ is the decoder's hidden state after masked self-attention in layer l (shape `(B, T_tgt, d_model)`), and H_{enc} is the encoder's final output (shape `(B, T_src, d_model)`).

What this means: each decoder position computes attention over all encoder positions. The attention weight matrix has shape `(B, h, T_tgt, T_src)` — each target position assigns a weight to each source position. The decoder "asks questions" via its queries, and the encoder "answers" through its keys and values.

Consider translating "The cat sat on the mat" to "Die Katze saß auf der Matte." When the decoder is generating "Katze" (German for "cat"), the query vector at that position will, through the attention mechanism, produce high attention weights on "cat" in the encoder output. The decoder is learning a soft alignment between source and target.

There is **no causal mask** in cross-attention — every decoder position can attend to every encoder position. The entire input is known (the encoder has already processed it), so there is nothing to hide. The only mask applied in cross-attention is the source padding mask, same as in the encoder's self-attention.

This cross-attention mechanism is conceptually the same as Bahdanau attention from the RNN seq2seq world: the decoder attends to the full encoder output when generating each token. But here it is multi-headed (8 heads in the original paper) and computed without any recurrence, making it far more parallelizable.

5 Embeddings

Token embeddings are a learned lookup table that maps each token ID (an integer) to a dense vector of size d_{model}:

Conceptual — illustrating the idea

```
embedding = nn.Embedding(vocab_size, d_model)
# Input: (B, T) tensor of integer token IDs
# Output: (B, T, d_model) tensor of embedding vectors
```

Each row of the embedding matrix corresponds to one token in the vocabulary. These vectors are initialized randomly and learned during training. After training, tokens with similar meanings will tend to have similar embedding vectors.

The original paper uses a **shared embedding matrix** for both the encoder and decoder. This makes sense when the source and target languages share a vocabulary (common with subword tokenization schemes like BPE, which construct a joint vocabulary across both languages).

BPE is covered in detail in Document 05, Section 1. Sharing means the model uses one set of embedding vectors for, say, the token "the" regardless of whether it appears on the source side or the target side.

Tip: The $\sqrt{d_{\text{model}}}$ scaling. The paper multiplies embedding vectors by $\sqrt{d_{\text{model}}}$ before adding positional encodings. This is easy to miss but has a concrete reason. `nn.Embedding` initializes its weight matrix from a standard normal distribution, which gives vectors with entries roughly distributed as $\mathcal{N}(0, 1)$. The L2 norm of such a d_{model}-dimensional vector is approximately $\sqrt{d_{\text{model}}}$, meaning each component has magnitude roughly 1. But in practice, many implementations (including the paper's) initialize with variance closer to $1/d_{\text{model}}$, which gives per-component magnitudes of roughly $1/\sqrt{d_{\text{model}}}$. Meanwhile, the sinusoidal positional encodings (described next) have components bounded in $[-1, 1]$, so their per-component magnitude is ~1. Without scaling, the positional encodings would dominate the token embeddings. Multiplying by $\sqrt{d_{\text{model}}}$ brings the embedding magnitudes up to the same scale as the positional encodings, so neither overwhelms the other.

6 Positional Encoding

Attention is **permutation-invariant**: if you shuffle the input tokens, the set of output vectors is the same set (just shuffled the same way). Formally, for any permutation π:

$$\text{Attention}(\{x_1, x_2, x_3\}) = \text{Attention}(\{x_{\pi(1)}, x_{\pi(2)}, x_{\pi(3)}\})$$

Without positional information, "the cat sat on the mat" and "mat the on sat cat the" would produce identical representations. Word order clearly matters, so we must inject position information.

The original paper uses **fixed sinusoidal positional encodings**:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

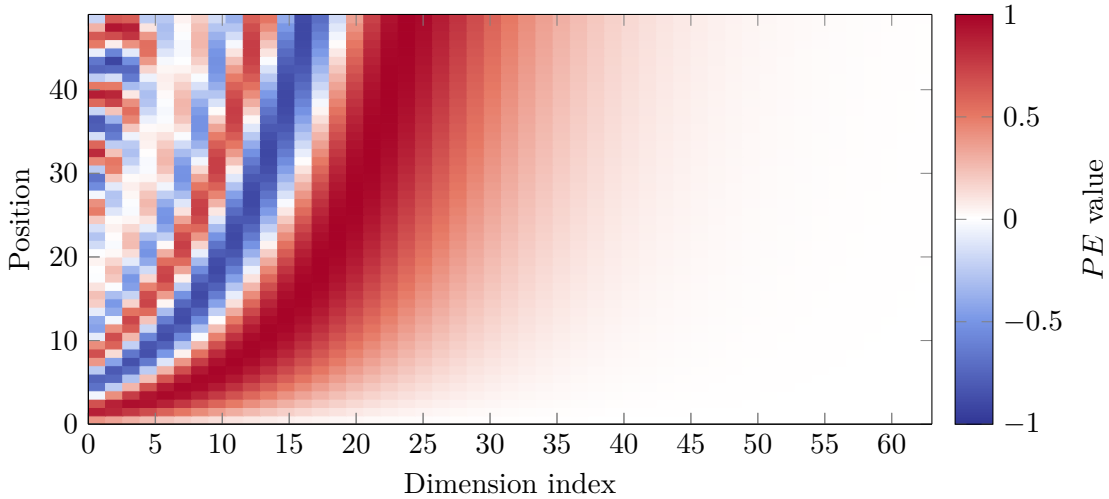
where pos is the position in the sequence (0, 1, 2, ...) and i is the dimension pair index (0, 1, 2, ..., $d_{\text{model}}/2 - 1$).

Tip: Reading the two formulas. The goal is to build a matrix PE of shape (T, d_{model}) — same shape as the token embeddings — so we can add them element-wise. The two formulas fill this matrix column by column: the first formula fills even columns $(0, 2, 4, \ldots)$ with sin, the second fills odd columns $(1, 3, 5, \ldots)$ with cos. Columns $2i$ and $2i{+}1$ form a pair that share the same frequency but use sin and cos respectively — like the x and y coordinates of a point on a circle. Each pair oscillates at a different frequency controlled by i: pair $i{=}0$ oscillates fast, and the last pair oscillates extremely slowly. The result is that each row (position) gets a unique combination of fast and slow oscillations across its d_{model} dimensions — a unique positional fingerprint. This is the matrix that gets added: `x = token_embeddings + PE`.

Each pair of dimensions $(2i, 2i+1)$ forms a sinusoid at a different frequency:

- Dimensions 0 and 1: wavelength $= 2\pi$ (the fastest oscillation — the encoding changes rapidly across positions)
- Last pair of dimensions: wavelength $= 2\pi \times 10000$ (extremely slow — the encoding barely changes from one position to the next)

The heatmap below shows the positional encoding matrix for 50 positions and 64 dimensions. Each row is one position, each column is one dimension, and color maps to value (-1 to $+1$). The left columns oscillate rapidly across positions (high frequency); the right columns change very slowly (low frequency). Each row has a unique pattern — the positional fingerprint.



The range of frequencies is geometric, spanning from 2π to $2\pi \times 10000$. This creates a unique "fingerprint" for each position: no two positions have the same pattern of values across all dimensions.

The geometric interpretation. Each dimension pair $(2i, 2i+1)$ can be viewed as a point on a circle rotating at frequency $1/10000^{2i/d_{\text{model}}}$. As position increases, the point traces the circle. The key property is that $PE(pos + k)$ can be expressed as a linear transformation of $PE(pos)$ for any fixed offset k:

$$PE(pos + k) = M_k \cdot PE(pos)$$

where M_k is a rotation matrix that depends only on k, not on pos. This means the model can learn to compute relative positions using linear operations on these encodings.

Why sinusoidal and not learned? The paper argues that sinusoidal encodings may generalize to sequence lengths longer than those seen during training, since the sinusoidal functions are

defined for any position. In practice, learned positional embeddings (as used in GPT-1 and GPT-2) work just as well for fixed-length contexts and are simpler to implement. Our BabyGPT in Document 04 will use learned positional embeddings.

A PyTorch implementation of sinusoidal positional encoding:

Demo — try in a REPL

This function is not part of our project — our GPT uses learned positional embeddings instead. Try it to visualize the encoding patterns.

```
import torch
import math

def sinusoidal_positional_encoding(max_len: int, d_model: int) ->
    torch.Tensor:
    """Compute sinusoidal positional encodings.

    Returns:
        (max_len, d_model) tensor of positional encodings.
    """
    pe = torch.zeros(max_len, d_model)
    position = torch.arange(0, max_len, dtype=torch.float).unsqueeze(1)
        # (max_len, 1)
    # Compute the denominator in log-space for numerical stability:
    # 10000^(2i/d_model) = exp(2i/d_model * ln(10000))
    div_term = torch.exp(
        torch.arange(0, d_model, 2, dtype=torch.float) *
            (-math.log(10000.0) / d_model)
    )  # (d_model/2,)

    pe[:, 0::2] = torch.sin(position * div_term)  # even dimensions
    pe[:, 1::2] = torch.cos(position * div_term)  # odd dimensions
    return pe
```

The result is added element-wise to the token embeddings: `x = token_embedding * sqrt(d_model) + positional_encoding`. These encodings are not learned — they are computed once and remain fixed throughout training.

7 The Output Head (Generator)

After the stack of 6 decoder layers, we need to convert the decoder's output vectors (shape `(B, T_tgt, d_model)`) into a probability distribution over the vocabulary. This happens in two steps:

1. A **linear layer** projects from d_{model} to `vocab_size`: the output is a vector of raw scores (logits) for each token in the vocabulary.
2. **Softmax** converts these logits into a probability distribution.

Conceptual — illustrating the idea

```
# After decoder stack:
logits = linear(decoder_output)  # (B, T_tgt, vocab_size)
probs = softmax(logits, dim=-1)  # (B, T_tgt, vocab_size)
```

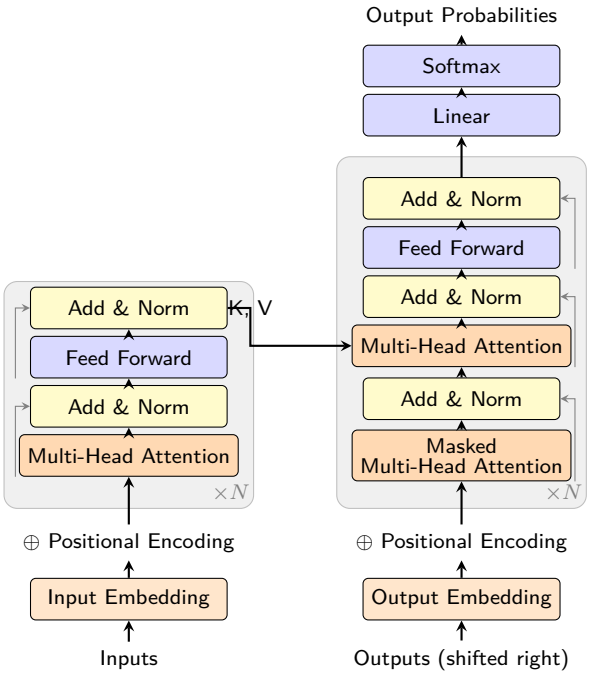
Weight tying. The original paper shares the weight matrix between the token embedding layer and this output projection layer. The embedding matrix has shape `(vocab_size, d_model)`. The output linear layer needs a weight matrix of shape `(d_model, vocab_size)` — which is exactly the transpose of the embedding matrix.

Why does this work? Consider what each does: the embedding layer maps a token ID to a vector in d_{model}-dimensional space. The output layer maps a d_{model}-dimensional vector to a score for each token in the vocabulary. When we use the transposed embedding matrix for the output projection, the score for a given token is the dot product between the decoder's output vector and that token's embedding vector. Tokens whose embeddings are similar to the decoder's output get high scores. This is a natural and consistent choice: the model's notion of "what a token means" (the embedding) is the same representation it uses to predict the next token.

Weight tying also has a practical benefit: for large vocabularies (32K or more tokens), the embedding matrix is a significant fraction of the model's total parameters. Sharing it with the output layer saves that entire parameter count.

8 Putting It All Together

The complete architecture, following the layout from "Attention Is All You Need" Figure 1:



Each `EncoderLayer` ($\times N$ on the left) contains:

- Multi-head self-attention (bidirectional) + residual + LayerNorm
- Feed-forward network + residual + LayerNorm

Each `DecoderLayer` ($\times N$ on the right) contains:

- Masked multi-head self-attention (causal) + residual + LayerNorm
- Multi-head cross-attention (Q from decoder, K/V from encoder) + residual + LayerNorm
- Feed-forward network + residual + LayerNorm

The original paper's hyperparameters:

Parameter	Base Model	Big Model
Layers (N)	6	6
d_{model}	512	1024
d_{ff} (FFN hidden dim)	2048	4096
Attention heads (h)	8	16
$d_k = d_v$ (per-head dim)	64	64
Total parameters	~65M	~213M

Training details:

- **Hardware:** 8 NVIDIA P100 GPUs
- **Training time:** ~3.5 days for the base model, ~12 hours per training step on 8 GPUs
- **Optimizer:** Adam ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-9}$) with a custom learning rate schedule:

$$lr = d_{\text{model}}^{-0.5} \cdot \min(step^{-0.5},\ step \cdot warmup_steps^{-1.5})$$

This schedule linearly increases the learning rate for the first `warmup_steps` (4000 steps), then decreases it proportionally to $1/\sqrt{step}$. The warmup prevents large, destabilizing updates early in training when the model's parameters are still random.

- **Regularization:** dropout of 0.1 applied to sublayer outputs, attention weights, and embeddings. Label smoothing of 0.1 (instead of training on hard 0/1 targets, the model trains on targets where the correct class gets probability 0.9 and the remaining 0.1 is spread across all other classes — this hurts perplexity slightly but improves BLEU score and generalization).

Results: On the WMT 2014 English-to-German translation benchmark, the base model achieved 27.3 BLEU and the big model achieved 28.4 BLEU — state of the art at the time, surpassing all previous models including ensembles.

9 Looking Ahead

The encoder-decoder architecture is well-suited for sequence-to-sequence tasks where the input and output are distinct sequences (translation, summarization). But for **language modeling** — the task of predicting the next token given preceding tokens — the encoder and cross-attention are unnecessary overhead. There is no separate "input" to encode; the model just reads text and predicts what comes next.

GPT (Radford et al., 2018) simplifies the transformer by keeping only the decoder (with its causal mask), removing the encoder and cross-attention entirely. What remains is a stack of transformer blocks, each containing masked self-attention and a feed-forward network — exactly the blocks we built in Document 02.

Document 04 makes this simplification explicit and assembles our BabyGPT.